

SOPADMIN

Sistema de Ocorrências Públicas

Projeto: flysop

Documento de Funcionalidades do Sistema

1 Visão Geral do Sistema

O **SOPADMIN** é um sistema web desenvolvido em **Laravel 8 (PHP)** com o objetivo de registrar, gerenciar e monitorar **ocorrências públicas** em tempo real. A plataforma possui arquitetura **multi-tenant**, com módulos de administração, área exclusiva para motoristas e API pública para integração com aplicativos e portais.

1.1 Tecnologias e Infraestrutura

Backend	Laravel 8, PHP 7.3/8.0, Eloquent ORM
Banco de dados	PostgreSQL (consultas ILIKE, índices e FKs)
Autenticação de API	Laravel Sanctum (tokens pessoais)
Interface administrativa	AdminLTE 3 (blade)
Tempo real	Laravel Broadcasting + Pusher / Echo
Armazenamento de arquivos	AWS S3 (flysystem)
E-mails	GuzzleHTTP, Mail (Mailable/Notify)
Debug/observabilidade	Laravel Telescope
Deploy	Fly.io (fly.toml), Docker Compose, Procfile

1.2 Estrutura da Aplicação

- **Admin:** painel administrativo completo com CRUDs e dashboard operacional.
- **Driver:** painel específico para motoristas (web) com fluxo de aceitação e rastreamento.
- **Site:** página pública/landing (login e informações).
- **API:** endpoints REST com autenticação Sanctum para apps e integrações.
- **Tenant:** isolamento de dados por empresa/cliente (multi-tenant).

2. Módulo de Ocorrências (Núcleo do Sistema)

O módulo central permite o registro e o acompanhamento de ocorrências, com dados do cidadão, endereço, geolocalização, órgão emissor, tipo e status, além de anexos (fotos/documentos).

2.1 Dados da Ocorrência

- Título e descrição; nome, CPF, RG, e-mail e telefone do envolvido.
- Endereço completo e coordenadas (latitude/longitude) para exibição em mapa.
- Vínculos: usuário responsável, cliente, **órgão emissor (issuings)**, **tipo** e **status**.
- Atribuição de **motorista** responsável pelo atendimento.
- Anexos: upload de múltiplas imagens/arquivos por ocorrência.

2.2 Fluxo de Status

O sistema suporta o fluxo completo do atendimento, do registro à finalização:

Aberta	Ocorrência registrada, aguardando tratamento	Admin + Motorista
Aguardando aceitação	Atribuída a um motorista, aguardando aceite	Motorista + Admin
Aceita	Motorista aceitou a ocorrência	Motorista + Admin
Recusada	Motorista recusou a ocorrência	Motorista + Admin
Em deslocamento	Motorista a caminho do local	Motorista + Admin
Em atendimento	Motorista no local realizando o atendimento	Motorista + Admin
Finalizada	Atendimento concluído	Motorista + Admin

2.3 Funcionalidades do Admin

- CRUD completo: criar, editar, visualizar e excluir ocorrências.
- Listagem paginada com **busca** por título/nome.
- Atribuição de motorista (muda status automaticamente para “Aguardando aceitação”).
- Upload de anexos ao criar/editar; exclusão dos arquivos do disco ao remover.
- Tela de detalhe com mapa, timeline e status.

3. Área do Motorista

Painel específico para motoristas com acesso diferenciado. O motorista visualiza apenas as ocorrências atribuídas a ele (**driver_id**) ou ao seu órgão (**issuings_id**), podendo aceitar, recusar e atualizar o status do atendimento.

3.1 Cadastro e Situação do Motorista

Disponível	Motorista livre para receber ocorrências
Em deslocamento	Motorista a caminho de uma ocorrência
Em atendimento	Motorista realizando o atendimento no local

3.2 Fluxo de Trabalho do Motorista

- **Dashboard:** cards com ocorrências aguardando aceitação, em andamento e finalizadas.
- **Lista de ocorrências:** ocorrências vinculadas a ele ou ao órgão, com busca e paginação.
- **Detalhes:** dados completos, mapa, imagens e ações contextuais.
- **Aceitar/Recusar:** altera o status para “Aceita” ou “Recusada” (com validação de propriedade).
- **Atualizar status:** em deslocamento -> em atendimento -> finalizada.

3.3 Controle de Acesso

- Middleware **ensure.driver** restringe o grupo de rotas **/driver/***.
- Perfil/permisões de motorista no ACL (index, show, accept, reject, update-status).
- Admin pode operar o fluxo em nome do motorista (aceitar/atualizar).
- Validações: motorista só atua em ocorrências que lhe pertencem.

4. Rastreamento e Monitoramento em Tempo Real

O sistema permite acompanhar a posição GPS do motorista e a rota percorrida durante o atendimento, com atualização no painel administrativo via polling e/ou WebSocket.

4.1 Captura de Posição (Motorista)

- Botão “Compartilhar minha rota” na tela de detalhe da ocorrência.
- Uso da **Geolocation API** (navegador) via **watchPosition**, com envio periódico.
- Envio via **POST /driver/position** (latitude, longitude, occurrence_id, accuracy).
- **Rate limit** de 1 requisição a cada 3 s por usuário (anti-spam).
- Validação: a ocorrência informada deve pertencer ao motorista.

4.2 Armazenamento e Leitura (Admin)

driver_positions	Histórico de posições (driver, ocorrência, lat/lng, precisão, horário)
Rota do motorista	GET /admin/occurrences/{id}/driver-route — polyline da rota na ocorrência
Última posição	Última posição por motorista/ocorrência para o mapa
Evento em tempo real	DriverPositionUpdated via Broadcasting (canal por tenant/ocorrência)

4.3 Dashboard em Tempo Real

- **Ocorrências recentes:** endpoint JSON atualizado por polling, exibindo status e tipo.
- **Posições dos motoristas:** marcadores da última posição de cada motorista em deslocamento.
- Mapa com marcadores de ocorrências e motoristas; indicador de atualização “agora”.
- Mecanismo híbrido: **polling** (fallback) e **WebSocket/Broadcasting** (tempo real).

5. Painel Administrativo e Dashboard

O painel admin concentra o gerenciamento de todos os módulos e exibe indicadores operacionais no dashboard, com cards, gráficos e mapa das ocorrências.

5.1 Dashboard

Usuários	Total de usuários do tenant
Ocorrências	Total de ocorrências registradas
Categorias	Total de categorias cadastradas
Tipos de ocorrência	Total de tipos cadastrados
Roles / Permissões	Total de cargos e permissões do ACL
Mapa ao vivo	Marcadores de ocorrências e posição dos motoristas

5.2 Módulos Gerenciáveis

Ocorrências	CRUD, busca, anexos, atribuição de motorista, rota
Tipo de Ocorrência	Cadastro e busca dos tipos
Status de Ocorrência	Cadastro e busca dos status do fluxo
Órgãos Emissores (Issuings)	Cadastro e busca de órgãos
Motoristas	Cadastro, busca e vínculo com usuário
Usuários	Cadastro, busca, vínculo de roles e tenant
Empresas (Tenants)	Cadastro, busca, planos e assinatura
Roles / Perfis / Permissões	ACL completo com vínculos
Categorias / Produtos / Mesas	CRUDs com busca (tenant-aware)
E-mails	Envio de e-mails/teste a partir do painel

6. Controle de Acesso (ACL)

O sistema possui controle de acesso completo com a relação **Usuários <-> Roles (cargos) <-> Permissões e Perfis <-> Permissões**.

6.1 Entidades do ACL

- **Permissão:** ação específica (ex.: criar ocorrência, ver motoristas).
- **Role (cargo):** agrupa permissões e é vinculado a usuários.
- **Perfil:** conjunto de permissões atribuído a usuários.
- **Vínculos:** user<->role, role<->permission, profile<->permission.

6.2 Telas de Vínculo

Usuário <-> Roles	/users/{id}/roles, users/{id}/roles/create, attach/detach
Role <-> Permissões	/roles/{id}/permission, attach/detach
Perfil <-> Permissões	/profiles/{id}/permission, attach/detach
Permissão <-> Roles/Perfis	/permission/{id}/roles, /permission/{id}/profiles

7. Multi-Tenancy

O projeto adota arquitetura multi-tenant: cada **empresa (tenant)** possui seus usuários e dados isolados, com suporte a planos e assinaturas.

7.1 Entidades

- **Tenant:** CNPJ/CPF, nome, e-mail, URL, UUID, logo, status ativo.
- **Assinatura:** subscription, expiração, active/suspended, subscription_id.
- **Planos:** tabelas plans e details_plan com plan_profile (vínculo plano<->perfil).
- **Isolamento:** trait TenantTrait e scopes (ex.: scopeTenantUser) filtram por tenant.

7.2 Relacionamentos

User	belongsTo(Tenant) — usuários do tenant
Driver	belongsTo(Tenant) — motoristas do tenant
Client / ClientConsumer	tenant_id — clientes da empresa
Category / Product / Table	TenantTrait — escopo automático

8. API Pública (Laravel Sanctum)

A API REST permite autenticação de clientes e consultas/criação de ocorrências para apps móveis, portais e integrações externas.

8.1 Autenticação

POST	/api/sanctum/token	Autentica cliente (e-mail/senha) e retorna token
GET	/api/auth/me	Dados do cliente autenticado
POST	/api/auth/logout	Revoga os tokens do cliente
POST	/api/client	Registra novo cliente
POST	/api/clientConsumer	Registra cliente consumidor

8.2 Ocorrências e Demais Endpoints

GET	/api/occurrences	Lista paginada de ocorrências
GET	/api/occurrences/{id}	Detalhes + anexos da ocorrência
GET	/api/occurrences/getOccurrenceByClientId/{id}	Ocorrências por cliente
POST	/api/occurrences	Cria nova ocorrência (com anexos)
GET	/api/tenants	Lista de empresas/tenants
GET	/api/tenants/{uuid}	Detalhes do tenant por UUID
GET	/api/typeOccurrence	Lista de tipos de ocorrência
POST	/api/driver/position	Motorista envia posição GPS

9. Notificações e E-mail

- **Envio de e-mail:** notificação sobre ocorrência criada (MailNotify / MailController).
- **Fila:** envio de e-mails enfileirado para não bloquear a resposta (previsto no plano).
- **Notificação ao motorista:** aviso ao atribuir ocorrência (e-mail/push — previsto).
- **Notificação ao admin:** aviso quando motorista aceita/atualiza ocorrência (previsto).

10. Segurança e Boas Práticas

Autenticação	Guard web + Sanctum; senhas com Hash; tokens pessoais
Autorização	ACL (roles/perfis/permisões) e middleware ensure.driver
Isolamento	Escopo por tenant em modelos e consultas
Validação	Form Requests e regras de validação (ex.: status existe, ocorrência pertence)
Rate limit	throttle no envio de posição e ações do motorista
Arquivos	Upload validado e armazenado em S3/disco privado
Observabilidade	Laravel Telescope para requisições, jobs e falhas

11. Deploy e Infraestrutura

fly.toml / Dockerfile.fly / Procfile	Deploy na plataforma Fly.io
Dockerfile / Dockerfile.dev	Imagens para produção e desenvolvimento
docker-compose.dev.yml	Ambiente local com Docker Compose
Dockerfile.nginx	Servidor web Nginx
vercel.json	Configuração opcional para frontend na Vercel
Procfile	Processos (web, worker) para deploy em PaaS

12. Melhorias Planejadas (Roadmap)

Os documentos *PLANO_MELHORIAS_OCORRENCIAS.md*, *PLANO_ROTA_MOTORISTA_TEMPO_REAL.md* e *ARQUITETURA_MOTORISTA.md* definem o backlog com as seguintes prioridades:

12.1 Prioridade Alta

- Número único de ocorrência (ex.: OC-2025-00042) e campo de prioridade/urgência.
- Escopo por tenant em todas as consultas de ocorrências.
- Área do motorista completa (já implementada) com app mobile nativo (fase posterior).
- WebSockets/Broadcasting para atualização em tempo real sem polling.
- Timeline de alterações e filtros (status, tipo, data, motorista) no mapa e lista.
- API pública e formulário público (sem login) para o cidadão registrar ocorrência.
- Revisão de permissões por perfil e rate limit na API pública.

12.2 Prioridade Média / Baixa

- Rascunho de ocorrência, webhook de integração, fotos no create.
- Histórico de alterações (auditoria) e atribuição em massa de motoristas.
- Clustering no mapa, notificações no navegador e resumo diário por e-mail.
- Atalho “Nova ocorrência” no dashboard, PWA/offline e campos condicionais.
- Chat admin<->motorista, assinatura digital e relatórios de desempenho do motorista.

Documento gerado a partir da análise do código-fonte do projeto flysop.